

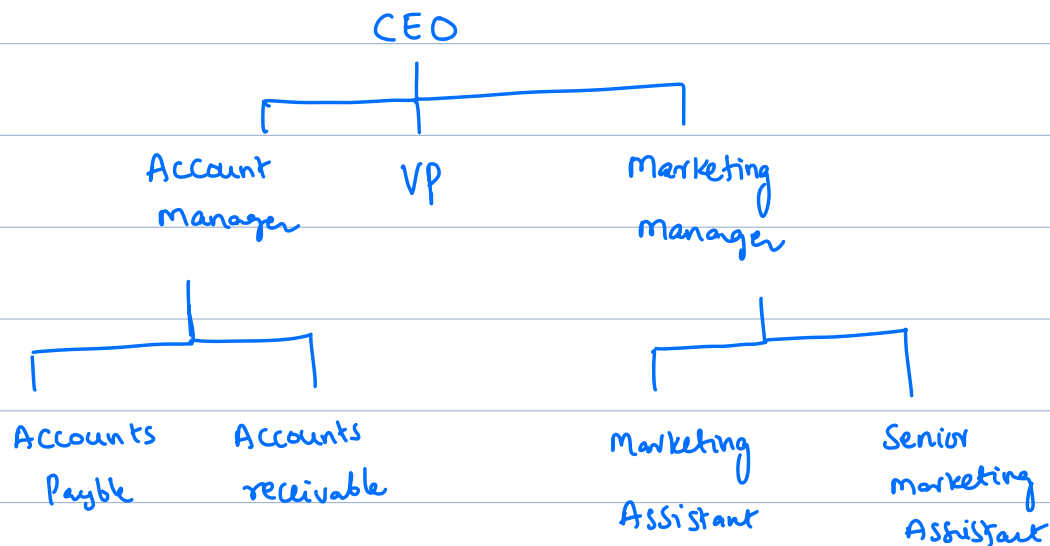
Trees

- A linear data structure is one in which data items are arranged sequentially or linearly such that each element is directly linked to its previous and the next elements.
- All the elements in a linear data structure can be traversed in a single run.
- Arrays, linked list, stack and queue are the different types of linear data structures.

In this chapter, we study about a non-linear data structure called a tree.

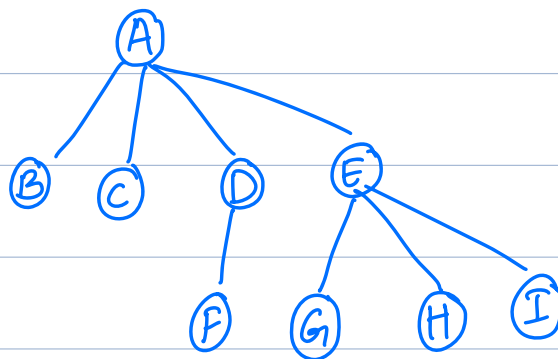
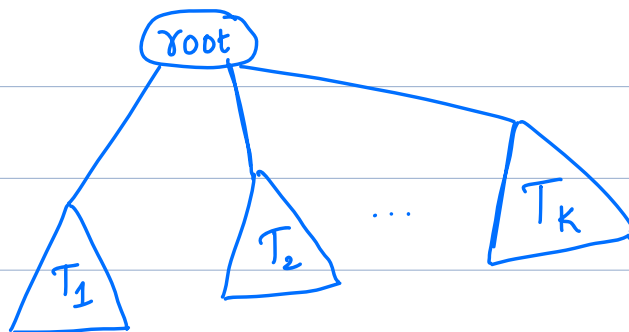
This is mainly used to represent data containing a hierarchical relationship between elements.

E.g. Family trees, table of Contents, Company organization Structure



Def.

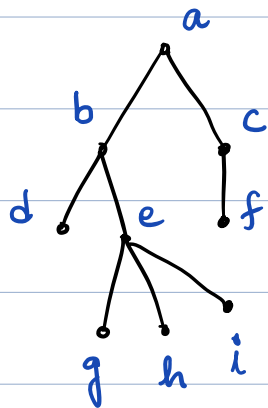
A tree is a collection of nodes. The collection can be empty. Otherwise a tree consists of a distinguished node r , called the **root**, and zero or more non-empty (sub) trees $T_1, T_2, \dots, T_k$ each of whose roots are connected by a directed edge from r .



Terminology:

- First node (starting node) is root
- Each node has variable number of references to successors (children)
- Each node (other than the root), has exactly one node as its predecessor (parent)
- Nodes with same parent are siblings
- Nodes with no children are called leaves

Example:



Refer to any
Standard textbook for
the terminology.

'a' : root node

b is the parent of d

d and e are children of b

d, g, h, f are leaves [leaf nodes [nodes with no child]]

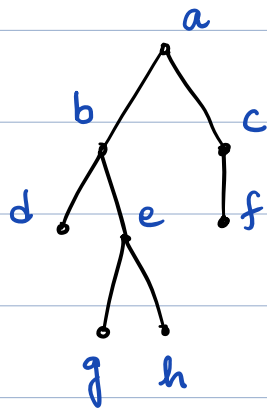
a, b, c, e are internal nodes [nodes with at least one child]

b is an ancestor of g

f is a descendant of a

Binary tree:

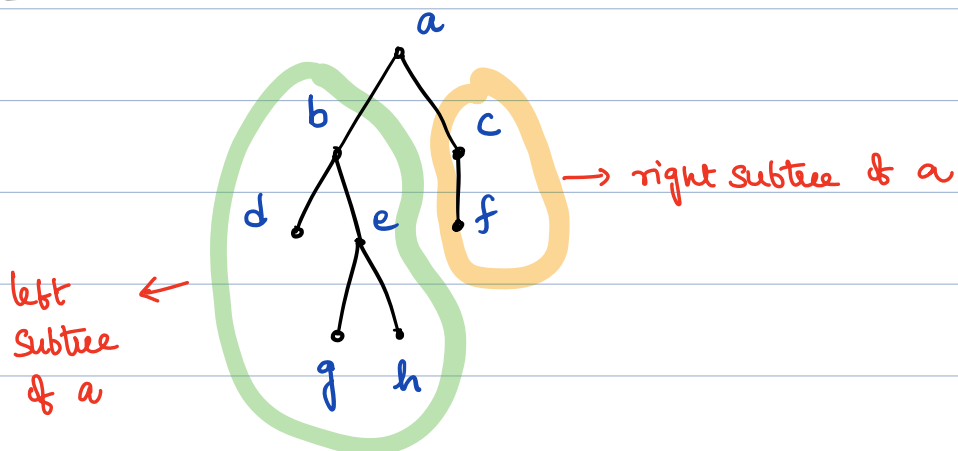
It is a tree in which every node/vertex has at most two children.



Binary tree T

- In the above figure 'a' is the root node

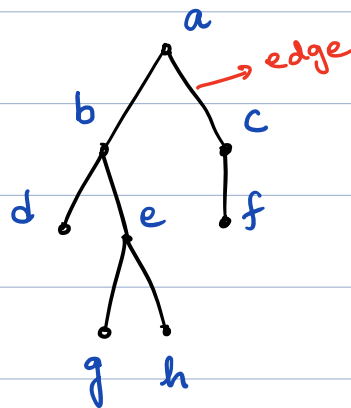
-



Edge: It is the line connecting a node to any of its children.

Path: A sequence of nodes such that any two consecutive nodes in the sequence have an edge.

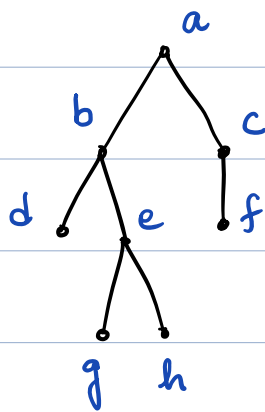
The path from a to g
is given by : a, b, e, g



Def:

The **height** of a node in a binary tree is the number of edges on the longest simple downward path from the node to a leaf. Height of the tree is height of the root.

Example:



① Height of the node g is zero and b is two

② Height of the tree is Three.

The depth of a node is the number edges from the node to the tree's root.

① Depth of the node e is two.

② Depth of root node is zero.

Level of node = Its depth + 1

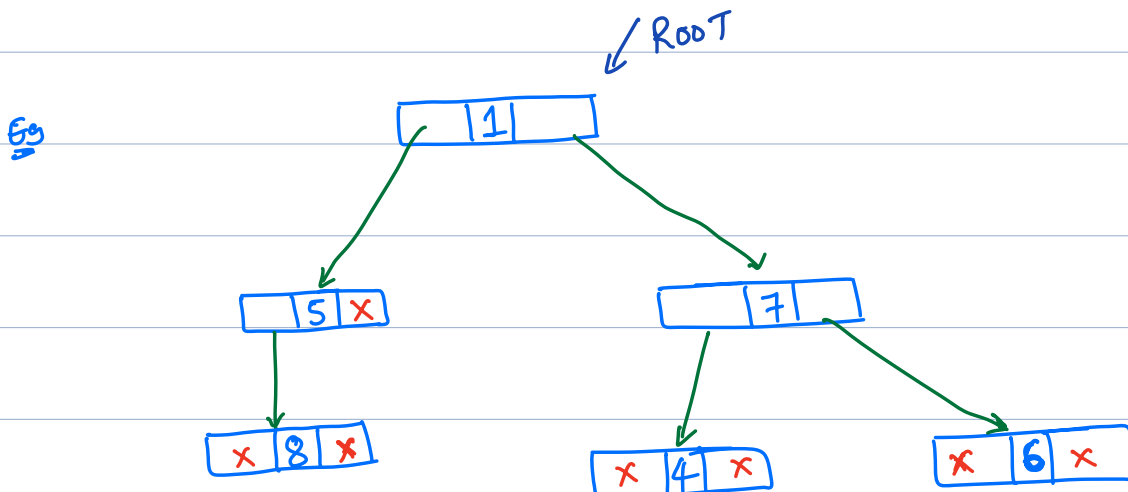
Representation of Binary trees

We can represent Binary trees using linked lists.

Every node will have three parts :

- a data element
- a Pointer to the left node
- a pointer to the right node.

Every binary tree has a pointer ROOT, which points to the root element of the tree. If $ROOT = NULL$ then the tree is empty.



Linked representation of a binary tree

Traversing a Binary tree

Traversing a binary tree is the process of visiting each node in the tree exactly once in a systematic way.

Since tree is non-linear data structure, there is no unique traversal

- ① Pre-order traversal
- ② In-order traversal
- ③ Post-order traversal

Pre-order traversal

To traverse a non-empty binary tree in pre-order the following operations are performed recursively at each node. the algorithm works by

- ① Visiting the root node
- ② traversing the left subtree
- ③ traversing the right subtree

Preorder (node)

If node == NULL

return ;

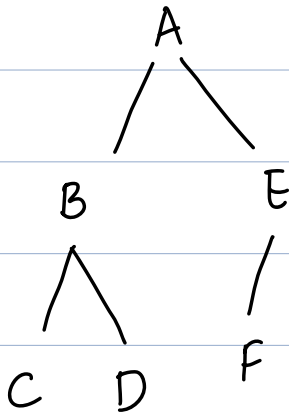
Print "node.data"

Preorder (node.left)

Preorder (node.right)

Initially
node contains
the address of
the root.

Eg



Pre-order traversal visits
the nodes in the following
order.

A B C D E F

In-order traversal

To traverse a non-empty binary tree in In-order the following operations are performed recursively at each node. the algorithm works by

- ① traversing the left subtree
- ② Visiting the root node
- ③ traversing the right subtree

In order (node)

If node == NULL

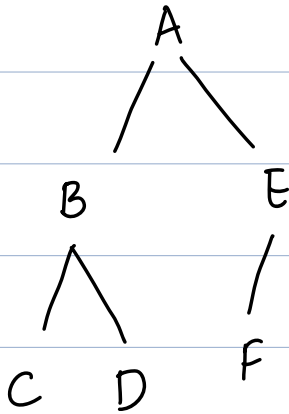
return ;

In order (node.left)

Print "node.data"

In order (node.right)

Eg



In-order traversal visits
the nodes in the following
order.

C B D A F E

Post-order traversal

To traverse a non-empty binary tree in post-order the following operations are performed recursively at each node. the algorithm works by

- ① traversing the left subtree
- ② traversing the right subtree
- ③ Visiting the root node

Post order (node)

If node == NULL

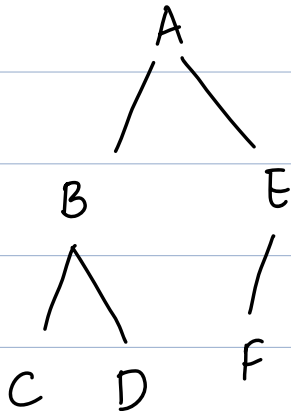
return ;

Post order (node.left)

Post order (node.right)

Print "node.data"

Eg



Post-order traversal visits
the nodes in the following
order.

CDBFEA

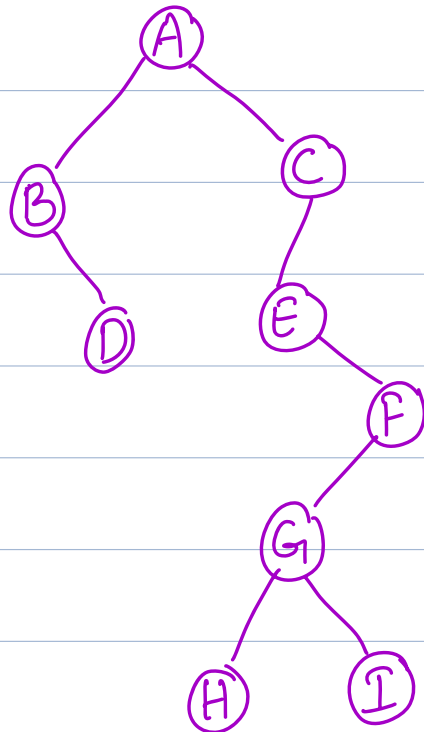
Practice Question:

For the following tree find the sequence of nodes that will be visited using

(a) Pre-order traversal

(b) In-order "

(c) Post-order "



Representing (implementing) rooted trees

One way to represent a tree would be to have in each node, besides its data, a link to each child of the node.

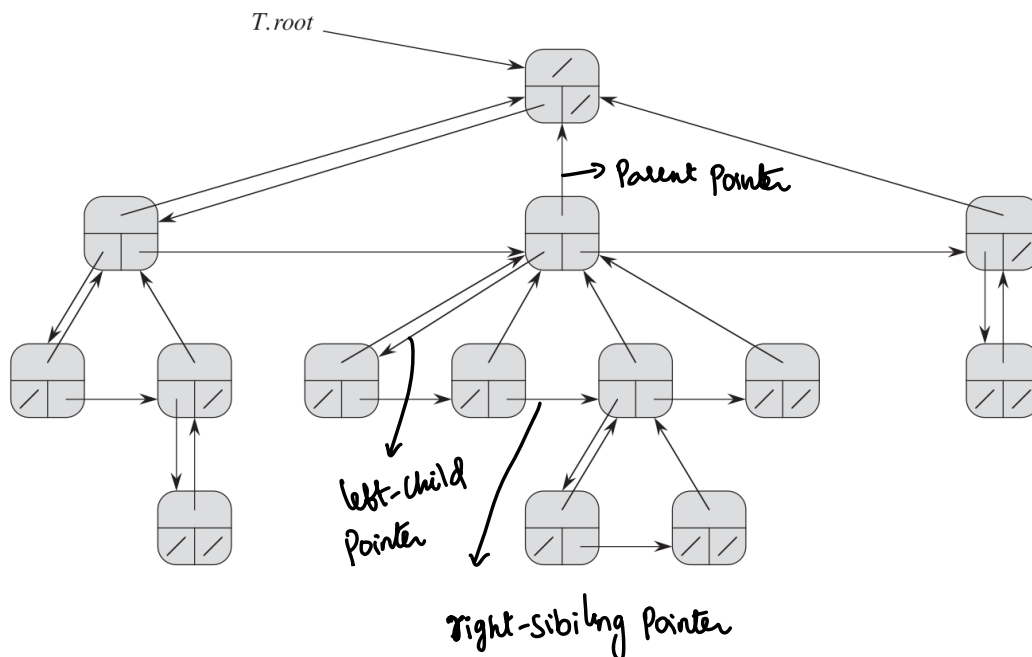
This representation might be infeasible when the number of children of a node is unbounded.

We use the **left-child, right-sibling** representation to represent trees.

Each node contains a **Parent Pointer** P , and $T.root$ points to the root of the tree T .

Instead of having a pointer to each of its children, however, each node x has only two pointers:

- $x.left-child$ Points to the leftmost child of node x
- $x.right-sibling$ Points to the sibling of x immediately to its right.



If a node x has no children then $x.\text{left-child} = \text{NIL}$
and if node x is the rightmost child of its Parent
then $x.\text{right-sibling} = \text{NIL}$.